MLflow, Explained

What it is — and exactly how we use it in this project

MLOps & Model Deployment

Vel Tech Rangarajan Dr. Sagunthala R&D Institute

6 – 8 August 2026

Dr. Venkateshwaran (Venkat) Loganathan

Senior Director of Engineering, ConverSight.ai

How to use this. Read sections 1–3 before Lab 1 — they explain why experiment tracking exists and where MLflow stores things. Keep sections 4–10 open beside you during the lab: section 4 walks through our actual `train.py`, and section 8 lists every error you are likely to hit, with the fix.

MLflow, Explained — and exactly how we use it

A student handout for Lab 1. Read the first two pages before the lab; keep the rest beside you while you work. Everything here refers to **our** project, `mlops-sample-project`.

---

1. The problem MLflow solves

You train a model. You change `n_estimators` from 100 to 300. It gets better. You change something else. It gets worse. Two hours later your manager asks:

> "*That model with 0.71 AUC — what settings produced it, and can you rebuild it?*"

If your answer lives in your terminal scrollback, a notebook cell you've overwritten, and your memory, **you cannot answer.** And a model you cannot rebuild is a model you cannot ship, debug, or defend.

MLflow is a lab notebook for machine learning that writes itself. Every time you train, it records what you set, what you got, and the model itself — automatically, in one place, permanently.

> **The interview version:** "*MLflow gives every training run an identity — parameters, metrics and artifacts stored together — so results are reproducible and comparable instead of anecdotal.*"

---

2. Four words you need

Term	What it means	In our project
Run	One execution of `train.py`, with its params, metrics and files	`youthful-lynx-308`
Experiment	A named group of related runs	`loan-default`
Artifact	Any file a run produces (the model, a plot, a CSV)	`model/`, `model.joblib`
Model Registry	A catalogue of *chosen* models, with version numbers	`loan-default-model` v1

A run is one attempt. An experiment is the whole set of attempts. The registry is where the **winner** gets an official name and version so other code can find it.

---

3. Where MLflow actually stores things — read this twice

MLflow splits its storage in two, and confusing them is the single most common way to lose an afternoon.

	`mlflow.db`	`mlruns/`
Called the	**backend store**	**artifact store**
Holds	params, metrics, run IDs, tags — the *numbers*	the actual *files*: models, plots
Format	one SQLite file	folders per run
If you delete it	you lose all run history	**you lose your models** — Register Model breaks

In `train.py` we choose the backend store explicitly:

```
MLFLOW_TRACKING_URI = "sqlite:///mlflow.db"   # a plain file in the project folder
EXPERIMENT_NAME = "loan-default"
```

No server, no cloud, no account. Just two things on disk.

> ⚠ **Therefore: the UI command must name the backend store**
>
> `bash mlflow ui --backend-store-uri sqlite:///mlflow.db --port 5000`
>
> A bare `mlflow ui` looks in the *default* location (`./mlruns` treated as a database) and finds no experiments. You'll get **"No Experiments Exist"** or a `MissingConfigException` traceback — while your runs sit safely in `mlflow.db`, unread. **Nothing is broken; you pointed the UI at the wrong place.**
>
> And **never** `rm -rf mlruns` to "clean up". That deletes the models you're about to register.

---

4. Our `train.py`, line by line

These are the only MLflow lines in the file. Find them and read them aloud with your pair.

Point at the store, name the experiment

```
mlflow.set_tracking_uri(MLFLOW_TRACKING_URI)   # use sqlite:///mlflow.db
mlflow.set_experiment(EXPERIMENT_NAME)         # group runs under "loan-default"
```

Created automatically the first time. That 30-line wall of database migration logs on your first run is MLflow building `mlflow.db` — **normal, and only once.**

Open a run

```
with mlflow.start_run() as run:
```

Everything logged inside this block belongs to this run. On exit it's sealed and given an ID. The random name (`youthful-lynx-308`) is MLflow being friendly — the **Run ID** is the real identifier.

Log the knobs you turned

```
mlflow.log_params({
    "model_type": "RandomForestClassifier",
    "n_estimators": n_estimators,
    "max_depth": max_depth,
    "test_size": test_size,
})
```

Params are inputs — your decisions. Log everything that could change the result, including `test_size`. A metric you can't attribute to a parameter is a metric you can't act on.

Log how well it did

```
metrics = {
    "accuracy": accuracy_score(y_test, preds),
    "f1":       f1_score(y_test, preds),
    "roc_auc":  roc_auc_score(y_test, proba),
}
mlflow.log_metrics(metrics)
```

Metrics are outputs — the scores. We log three deliberately, because one number hides things: on our data `accuracy ≈ 0.76` looks respectable while `f1 ≈ 0.40` reveals the model is weak at catching actual defaulters. **Never judge an imbalanced problem by accuracy alone.**

Log the model — twice, on purpose

```
mlflow.sklearn.log_model(model, artifact_path="model")   # (a) for the Model Registry
joblib.dump(model, MODEL_PATH)                           # (b) for the FastAPI app
mlflow.log_artifact(MODEL_PATH)                          # keep a copy with the run
```

Two formats because they have two different consumers:

- **(a) the MLflow model folder** — rich, self-describing, registry-ready. This is what you click **Register Model** on in Lab 1.
- **(b)** `model.joblib` — a single plain file, the simplest thing for **Lab 2's FastAPI app** and **Lab 3's Docker image** to load.

You will see this warning, and it is harmless:

```
WARNING mlflow.models.model: Model logged without a signature and input example.
```

MLflow is saying "*I don't know what input this model expects.*" In production you'd log a signature so wrong inputs are caught early — which is exactly the job we hand to **Pydantic in Lab 2**.

---

5. Reading the UI

Launch it in a **second terminal** (leave training in the first):

```
mlflow ui --backend-store-uri sqlite:///mlflow.db --port 5000
```

In Codespaces: click **Open in Browser** on the pop-up, or the **PORTS** tab → globe icon. *Not* `localhost:5000` — that's your laptop, not the Codespace. **On a laptop:** `http://127.0.0.1:5000`.

Then:

1. Click the `loan-default` experiment in the left sidebar.
2. ⚠ **Click `Columns ▾` and tick `roc_auc`, `f1`, `accuracy`, `n_estimators`, `max_depth`.** The default table shows only Run Name, Created, Duration and Source — **your metrics are hidden until you add them.** You cannot compare or sort until you do this.
3. Click a column header to **sort**. Tick several runs → **Compare**.
4. Click into a run to see **Parameters**, **Metrics** and **Artifacts**.

You can also query runs in code — useful for automation:

```
import mlflow
mlflow.set_tracking_uri("sqlite:///mlflow.db")
best = mlflow.search_runs(
    experiment_names=["loan-default"],
    order_by=["metrics.roc_auc DESC"],
).head(1)
print(best[["run_id", "metrics.roc_auc", "params.n_estimators"]])
```

---

6. The Model Registry — promoting a winner

Tracking answers "*what did I try?*" The registry answers "*which one is the model?*"

In the UI: open your best run → **Artifacts** → click the `model` folder → **Register Model** → name it `loan-default-model` → this creates **Version 1**.

Or in code:

```
mlflow.register_model(f"runs:/{run_id}/model", "loan-default-model")
```

Why this matters: serving code can then ask for a model **by name and stage**, never by filename.

```
# Nobody has to know which run or file this came from
model = mlflow.sklearn.load_model("models:/loan-default-model/Production")
```

That indirection is the whole point. **"Deploy the model" stops meaning "copy the right .pkl and hope" and starts meaning "promote version 4 to Production".** Rolling back becomes promoting version 3.

Each version keeps its **lineage** — the run, the params, the metrics that justified it. When someone asks in six months why this model is in production, the answer is a link.

7. Where MLflow touches the rest of the workshop

Lab	The connection
1	Log runs, compare them, register `loan-default-model` v1
2	FastAPI loads the `model.joblib` that `train.py` logged
3	Docker copies that same `model.joblib` into the image
4	CI re-runs training and the **accuracy gate** — the same metric MLflow records, now enforced. Our gate is **F1 ≥ 0.35**, set just under our measured **0.3958**
6	The drift monitor logs PSI and KS p-values to a second experiment, **`loan-default-monitoring`** — so *production health* is tracked exactly like training was

That last row is the mental leap worth taking: **tracking isn't a training-time tool, it's how you keep a system accountable for its whole life.**

8. When it goes wrong

Symptom	Cause	Fix
"No Experiments Exist"	UI started without `--backend-store-uri`	Relaunch with `--backend-store-uri sqlite:///mlflow.db`
`MissingConfigException: './mlruns/1/meta.yaml' does not exist` / "*Malformed experiment*"	Same cause — the UI is reading `mlruns/` as a database	Same fix. **Do not delete `mlruns/`**
Register Model fails or the artifact is empty	`mlruns/` was deleted	Re-run `python train.py` to regenerate artifacts
30 lines of `alembic` migration logs	First-ever run creating `mlflow.db`	Expected. Once only
`Model logged without a signature`	No input example given	Harmless — Lab 2 solves validation properly
No metrics in the run table	Columns hidden by default	**`Columns`** ▾ → tick the metrics
`Port 5000 is in use`	Something else has it (on macOS, AirPlay Receiver)	`--port 5001`, or turn AirPlay Receiver off
Runs vanished	You're in a different folder — the store is *relative*	`cd /workspaces/mlops-sample-project` first

9. What MLflow is not

Being clear about the boundary is what separates a confident answer from a hand-wave:

- **Not a scheduler.** It records runs; it doesn't decide when to train. (That's cron, Airflow, or CI.)
- **Not a monitoring system.** We *use* it to store drift metrics, but it won't alert you.
- **Not a feature store.** It versions models, not the data pipelines feeding them.
- **Not a substitute for git.** MLflow tracks *results*; git tracks *code*. You need both — and the run records which commit produced it.

---

10. Quick reference

```
# train + log a run
python train.py --n_estimators 300 --max_depth 10

# browse everything (SECOND terminal; keep it running)
mlflow ui --backend-store-uri sqlite:///mlflow.db --port 5000

# what exists on disk
mlflow.db      <- params & metrics  (the numbers)
mlruns/        <- artifacts         (the models) - DO NOT DELETE
model.joblib   <- plain model for FastAPI & Docker
```

Our project's names: experiment `loan-default` · registered model `loan-default-model` · monitoring experiment `loan-default-monitoring`.

Our baseline numbers: `accuracy 0.7583` · `f1 0.3958` · `roc_auc 0.7121`.

Three questions to check yourself

1. You deleted `mlruns/` but kept `mlflow.db`. What still works, and what doesn't?
2. Your teammate's UI shows no experiments but their `train.py` printed a Run ID. What did they do?
3. Why does `train.py` save the model **twice**, in two different formats?

(Answers: 1 — metrics and params still display, but artifacts are gone so you can't register or load a model. 2 — they launched `mlflow ui` without `--backend-store-uri`. 3 — the MLflow model folder is for the registry; `model.joblib` is the simple file FastAPI and Docker load.)